

```
// === ДОБАВЛЯЕМ ОБЯЗАТЕЛЬНУЮ ВЕРШИННУЮ ТОЧКУ ВХОДА ДЛЯ DIRECTX 11 ===
```

```
struct VS_OUTPUT
```

```
{
```

```
    float4 Pos : SV_POSITION;
```

```
    float2 Tex : TEXCOORD0;
```

```
};
```

```
VS_OUTPUT VS(float4 Pos : POSITION, float2 Tex : TEXCOORD)
```

```
{
```

```
    VS_OUTPUT output = (VS_OUTPUT)0;
```

```
    output.Pos = Pos;
```

```
    output.Tex = Tex;
```

```
    return output;
```

```
}
```

```
// =====
```

```
// ИСТИННЫЙ ОДНОПРОХОДНЫЙ ДЕФЕКТОСКОП АНАТОЛИЯ ФЕДОРОВИЧА (ОБРАЗЕЦ 2011 ГОДА)
```

```
// =====
```

```
// Входные ресурсы DirectX 11
```

```
Texture2D srcTexture : register(t0);    // Наш единственный сырой файл
```

```
Texture2D macroTexture : register(t1);  // Принимаем готовую макротекстуру блоков 46x46!
```

```
SamplerState samplerState0 : register(s0); // Point-сэмплер без замыливания
```

```
// Буфер констант (бывший register(c0))
```

```
cbuffer cbData : register(b0)
```

```
{
```

```
    float width;    // p0[0]
```

```
    float height;   // p0[1]
```

```
    float d_width;  // p0[2]
```

```
    float d_height; // p0[3]
```

```
    float splitX;   // Положение шторки
```

```

float padding;

float macroWidth; // Принимаем 46.0f от процессора
float macroHeight; // Принимаем 46.0f от процессора
};

static const float3 WEIGHT = float3(0.299f, 0.587f, 0.114f);

// Вспомогательная функция чтения и перевода пикселя в монохром (tex2D)
float GetGray(float2 uv)
{
    //float2 center_uv = uv + float2(padding, padding);
    //return dot(macroTexture.Sample(samplerState0, uv).rgb, WEIGHT);
    //return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
    return dot(srcTexture.Sample(samplerState0, uv).rgb, WEIGHT);
}

// 1. ШЕЙДЕР ДЛЯ ЛЕВОГО ВЕРХНЕГО УЧАСТКА
float4 func_LT(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

    float2 tex_in_3 = tex_in;

    // Раздвигаем соседей строго на физический размер макроблока 4x4
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    //float2 k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);

```

```

float k_LR = GetGray(tex_LR);
float k_TB = GetGray(tex_TB);
float k_LRTB = GetGray(tex_LRTB);

int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

float k_SS = ((k_S * (9 + i_x) + k_LR * (7 - i_x)) * (9 + i_y) +
              (k_TB * (9 + i_x) + k_LRTB * (7 - i_x)) * (7 - i_y)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 2. ШЕЙДЕР ДЛЯ ПРАВОГО ВЕРХНОГО УЧАСТКА
float4 func_RT(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

```

```

float2 tex_in_3 = tex_in;

// Раздвигаем соседей строго на физический размер макроблока 4x4
float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

//float2 k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
float k_S = GetGray(tex_in_3);
float k_LR = GetGray(tex_LR);
float k_TB = GetGray(tex_TB);
float k_LRTB = GetGray(tex_LRTB);

int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (9 + i_y * 2) +
              (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (7 - i_y * 2)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);

```

```
}
```

```
// 3. ШЕЙДЕР ДЛЯ ПРАВОГО НИЖНЕГО УЧАСТКА
```

```
float4 func_RB(float2 tex_in, float4 screen_pos)
```

```
{
```

```
    // Шаг дельты пикселя для полного кадра
```

```
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);
```

```
    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU  
    ===
```

```
    float2 tex_in_3 = tex_in;
```

```
    // Раздвигаем соседей строго на физический размер макроблока 4x4
```

```
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
```

```
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
```

```
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);
```

```
    //float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
```

```
    float k_S = GetGray(tex_in_3);
```

```
    float k_LR = GetGray(tex_LR);
```

```
    float k_TB = GetGray(tex_TB);
```

```
    float k_LRTB = GetGray(tex_LRTB);
```

```
    int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;
```

```
    int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;
```

```
    float k_SS = ((k_S * (15 - i_x * 2) + k_LR * (1 + i_x * 2)) * (15 - i_y * 2) +  
        (k_TB * (15 - i_x * 2) + k_LRTB * (1 + i_x * 2)) * (1 + i_y * 2)) / 256.0f;
```

```
    // Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
```

```
    float2 k1 = GetGray(tex_in);
```

```
    float delta = k1 - k_SS;
```

```

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// 4. ШЕЙДЕР ДЛЯ ЛЕВContextОГО НИЖНЕГО УЧАСТКА
float4 func_LB(float2 tex_in, float4 screen_pos)
{
    // Шаг дельты пикселя для полного кадра
    float2 d_xy = float2(d_width / 2.0f, d_height / 2.0f);

    // === КРИТИЧЕСКОЕ ИЗМЕНЕНИЕ: СЧИТАЕМ СОСЕДЕЙ ПРЯМО ОТ ТЕКУЩЕЙ ТОЧКИ, КАК НА CPU
    ===

    float2 tex_in_3 = tex_in;

    // Раздвигаем соседей строго на физический размер макроблока 4x4
    float2 tex_LR = tex_in + float2(d_width * 4.0f, 0.0f);
    float2 tex_TB = tex_in + float2(0.0f, d_height * 4.0f);
    float2 tex_LRTB = tex_in + float2(d_width * 4.0f, d_height * 4.0f);

    //float k1 = GetGray(tex_in + 2.0f * float2(d_width, d_height));
    float k_S = GetGray(tex_in_3);
    float k_LR = GetGray(tex_LR);
    float k_TB = GetGray(tex_TB);
    float k_LRTB = GetGray(tex_LRTB);

    int i_x = (int(round((tex_in.x + 4.0f * d_xy.x) * width)) % 4) * 2;

```

```

int i_y = (int(round((tex_in.y + 4.0f * d_xy.y) * height)) % 4) * 2;

float k_SS = ((k_S * (9 + i_x * 2) + k_LR * (7 - i_x * 2)) * (15 - i_y * 2) +
              (k_TB * (9 + i_x * 2) + k_LRTB * (7 - i_x * 2)) * (1 + i_y * 2)) / 256.0f;

// Шаг 3: Вычисляем "ту самую разность" (высокочастотные детали)
float2 k1 = GetGray(tex_in);
float delta = k1 - k_SS;

// Шаг 4 и 5: К средней яркости блоков применяется гамма-коррекция (пока 1.0f для теста)
float g_Gamma = 1.0f;
float k_SS_gamma = pow(saturate(k_SS), g_Gamma);

// Шаг 6: Прибавляем "ту разность" к новой яркости
float out_gray = saturate(k_SS_gamma + delta);

// Шаг 7: Выводим идеальный монолитный кадр на экран
return float4(out_gray, out_gray, out_gray, 1.0f);
}

// ГЛАВНЫЙ АППАРАТНЫЙ ШЕЙДЕР ФИЛЬТРАЦИИ
float4 PS_Stage3(VS_OUTPUT input) : SV_Target
{
    float2 tex0 = float2(input.Tex.x - 2.0f * d_width, input.Tex.y - 2.0f * d_height);

    // Получаем чистый, неплывущий номер макроблока 4x4 через целые числа экрана
    int block_x = int(input.Pos.x) / 4;
    int block_y = int(input.Pos.y) / 4;

    // Жестко и без сбоев округления выбираем нужную функцию зоны
    if (block_x % 2 == 0)
    {

```

```

        if (block_y % 2 == 0) return func_LT(tex0, input.Pos);
        else return func_LB(tex0, input.Pos);
    }
else
{
    if (block_y % 2 == 0) return func_RT(tex0, input.Pos);
    else return func_RB(tex0, input.Pos);
}
}

```

```

// ШЕЙДЕР СКВОЗНОГО КОПИРОВАНИЯ ЭКРАНА С УЧЕТОМ ШТОРКИ ДО/ПОСЛЕ
Texture2D statsTexture : register(t1); // Сюда подадим готовый фильтр из Stage3

```

```

float4 PS_Final(VS_OUTPUT input) : SV_Target
{
    if (input.Tex.x < splitX)
    {
        // Слева переводим сырой оригинал в честный монохром по вашим весам
        float4 src_color = srcTexture.Sample(samplerState0, input.Tex);
        float src_gray = dot(src_color.rgb, WEIGHT);
        return float4(src_gray, src_gray, src_gray, 1.0f);
    }

    // Тонкая разделительная черная линия
    if (abs(input.Tex.x - splitX) < 0.0015f) return float4(0.0f, 0.0f, 0.0f, 1.0f);

    // Справа выводим чистый результат дефектоскопа
    return statsTexture.Sample(samplerState0, input.Tex);
}

```

```

// Простой шейдер копирования кадра для ЭТАПА II
float4 PS_Copy(VS_OUTPUT input) : SV_Target

```

```
{  
    return srcTexture.Sample(samplerState0, input.Tex);  
}
```

// ВРЕМЕННАЯ ЗАГЛУШКА ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТА В MAIN.CPP

```
float4 PS(VS_OUTPUT input) : SV_Target  
{  
    return srcTexture.Sample(samplerState0, input.Tex);  
}
```

// === ОФИЦИАЛЬНЫЕ ЗАГЛУШКИ ДЛЯ ЧИСТОГО СТАРТА ПРОГРАММЫ БЕЗ ОШИБОК ===

```
float4 PS_Downsample2X(VS_OUTPUT input) : SV_Target  
{  
    return srcTexture.Sample(samplerState0, input.Tex);  
}
```

// === НАСТОЯЩЕЕ АППАРАТНОЕ СЖАТИЕ БЛОКОВ 4x4 В SHADERS.HLSL ===

```
float4 PS_Stage2(VS_OUTPUT input) : SV_Target  
{  
    float4 color_sum = float4(0.0f, 0.0f, 0.0f, 0.0f);
```

// Запускаем цикл по всему макроблоку 4x4 пикселя

```
for (int x = 0; x < 4; x++)
```

```
{
```

```
    for (int y = 0; y < 4; y++)
```

```
    {
```

// Вычисляем точный сдвиг для каждого из 16 пикселей внутри блока

// d_width и d_height — это наши шаги константного буфера

```
float2 uv_offset = input.Tex + float2(x * d_width, y * d_height);
```

```
  
    // Суммируем цвета всех точек
```

```

        color_sum += srcTexture.Sample(samplerState0, uv_offset);
    }
}

// Честно делим сумму на 16 пикселей, получая идеальное среднее значение фона!
return color_sum / 16.0f;
}

float4 PS_PixelShaderStage2(VS_OUTPUT input) : SV_Target
{
    float4 color_sum = float4(0.0f, 0.0f, 0.0f, 0.0f);
    for (int x = 0; x < 4; x++)
    {
        for (int y = 0; y < 4; y++)
        {
            float2 uv_offset = input.Tex + float2(x * d_width, y * d_height);
            color_sum += srcTexture.Sample(samplerState0, uv_offset);
        }
    }
    return color_sum / 16.0f;
}

```